

Hybrid Resource Analysis in Resource Aware ML 2: Inferring Logarithmic Cost Bounds on ML Programs

Arnav Sabharwal, Carnegie Mellon University. Email: arnavsab@andrew.cmu.edu

Overview

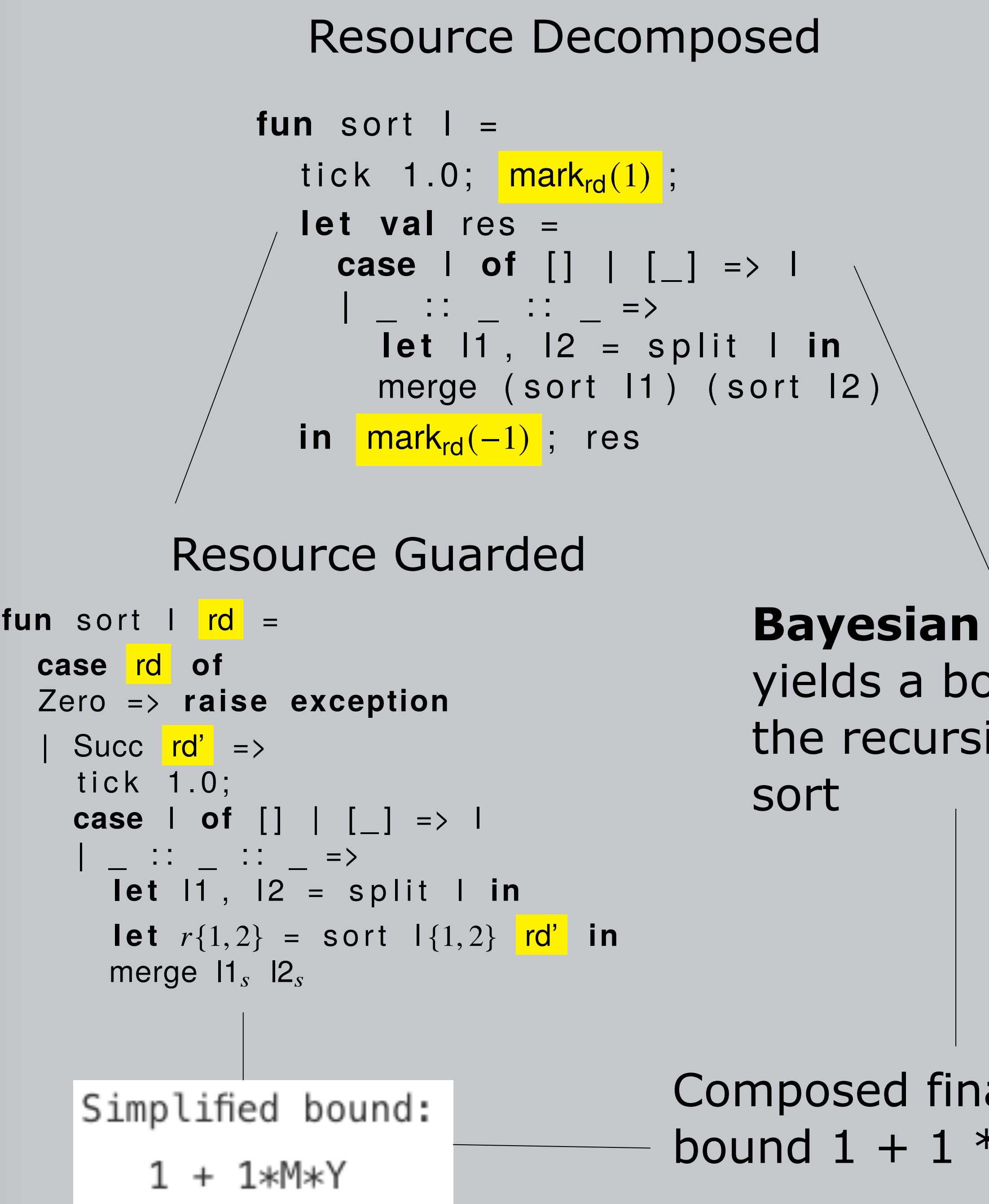
Given a program P , the goal of resource analysis is to derive a **sound** bound $f(x)$ on its resource use (e.g., time, space, stack depth, etc) in terms of its input x . Resource Aware ML (RaML) [2] is a tool based on Automatic Amortized Resource Analysis (AARA) [1] for deriving sound, **polynomial** cost bounds on purely functional ML programs.

```
fun sort l =
  tick 1.0 ;
  case l of [] | [_] => l
  | _ :: _ :: _ =>
    let l1, l2 = split l in
      merge (sort l1) (sort l2)
```

Simplified bound:
 $1 - 0.666667 * M + 1.33333 * M^2$

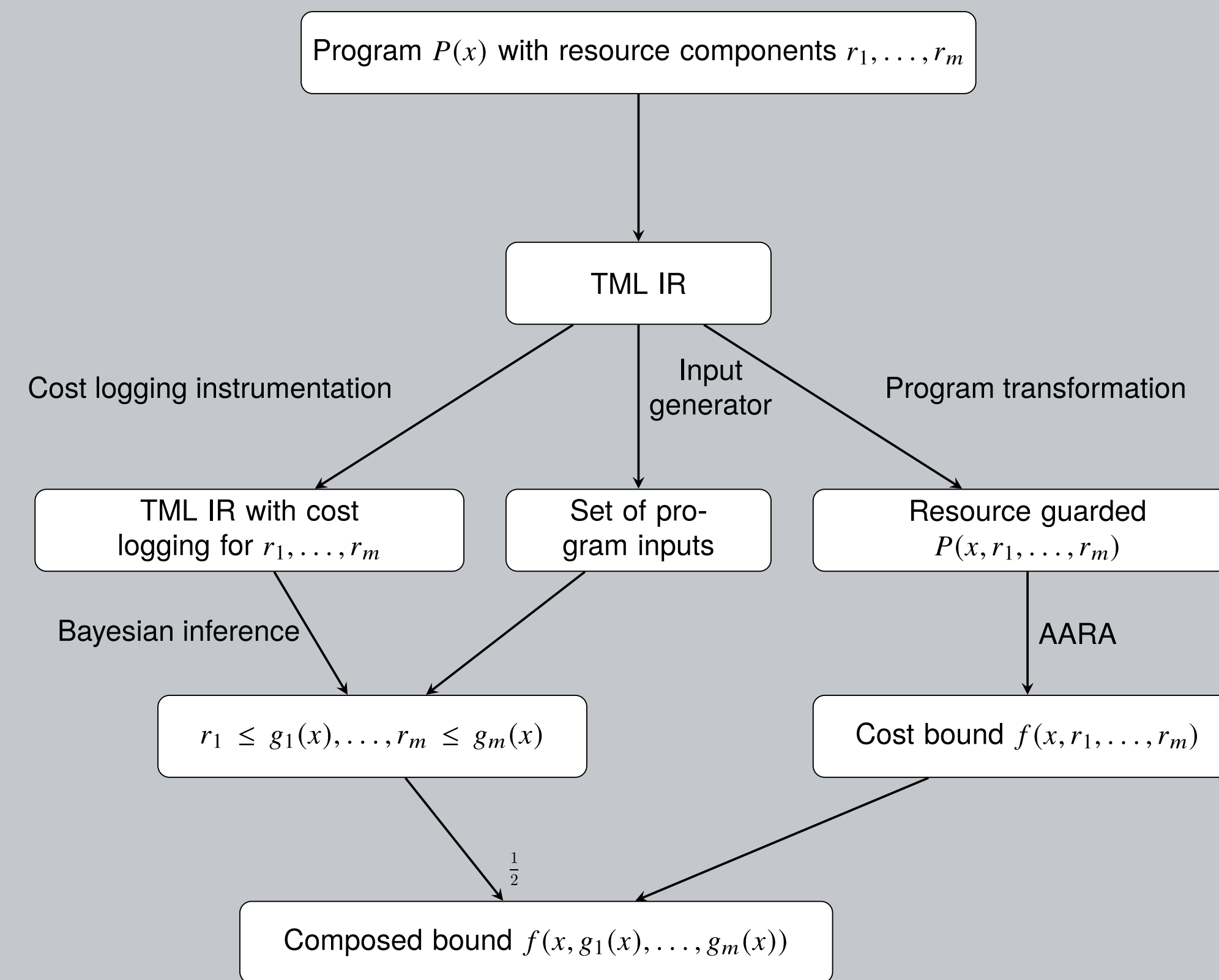
RaML is incapable of statically inferring bounds containing logarithms, e.g. mergesort. This is an instance of the incompleteness of static resource analyses. Dually, data-driven resource analyses are unsound.

Resource decomposition [3] is a hybrid resource analysis that combines the complementary strengths and weaknesses of RaML and data-driven resource analysis.



Contribution: the implementation of resource decomposition as a largely automated process in **RaML 2** [4], an improvement to RaML.

Methodology



Program Transformation

- Expressions e using mark are *effectful*, and $\Gamma \vdash e : \text{eff}$ identifies effectful expressions.
- Since TML doesn't make a type distinction between pure and effectful expressions, modify TML as follows to define TML^F :

Value types $\tau := 1 \mid \tau_1 \times \tau_2 \mid \tau_1 + \tau_2 \mid \text{rec}(t.\tau_1) \mid \tau \rightarrow C$

Monadic type $C := F(\tau)$

$$\frac{z \in \mathbb{Z} \quad \Gamma \vdash e : \rho}{\Gamma \vdash \text{mark}_s(z) ; e : \rho} \mapsto \frac{z \in \mathbb{Z} \quad \Gamma \vdash e : F(\rho)}{\Gamma \vdash \text{mark}_s(z) ; e : F(\rho)}$$

- Translate TML to TML^F via tr using the invariants:
 - $\Gamma \vdash e : \tau \wedge e \text{ pure} \implies \|\Gamma\| \vdash \text{tr}(e) : \|\tau\|$
 - $\Gamma \vdash e : \tau \wedge e \text{ eff} \implies \|\Gamma\| \vdash \text{tr}(e) : F(\|\tau\|)$
- Finally, assuming the TML program P contains m resource components, transform the $\text{TML}^F \text{tr}(P)$ into resource guarded TML $\llbracket \text{tr}(P) \rrbracket$ using the rule: $\llbracket F(\tau) \rrbracket = \mathbb{N}^m \rightarrow \mathbb{N}^m \times \llbracket \tau \rrbracket$.

$$\|\text{mark}_l(z) ; e\| = \langle r_1, \dots, r_m \rangle \mapsto \begin{cases} \|e\| \langle \dots, (r_l - z), \dots \rangle & r_l \geq z \\ \text{raise exception} & \text{otherwise} \end{cases}$$

Data-driven Analysis

- The generator takes as input a monomorphic, polynomial recursive type τ and a number N . It then generates a set X of N values $v : \tau$ on which program P can be run.
- For each $f : \tau \rightarrow \rho$ whose recursion depth we wish to bound, the user must define a function $s_f : \tau \rightarrow \mathbb{N}^k$, which maps values of type τ to a multi-dimensional notion of size.
- Program P is transformed to P' such that for any function f whose depth we wish to bound, for any $x \in X$, when f is called for the i^{th} time with argument v_i in the evaluation of $P'(x)$, we log $(d_i, s_f(v_i))$, where d_i is the peak recursion depth of f in the evaluation of $f(v_i)$.
- Finally, by implementing the Bayesian inference model of [3] in STAN [5], we infer a univariate linear or logarithmic recursion depth bound on each function f of interest.

Evaluation

We empirically demonstrate how RaML 2 with resource decomposition is able to infer tight cost bounds on common functional programs containing one or more functions with logarithmic recursion depth.

Benchmark	Ground Truth	Vanilla RaML	Resource Decomposition
MergeSort	$O(x \log x)$	$1.5 x ^2 - 1.5 x $	$O(x \log x)$
HeapInsert	$O(x \log x)$	$0.5 x ^2 + 0.5 x $	$O(x \log x)$
BalancedBST	$O((x_1 + x_2) \log x_1)$	$0.4 x_1 + x_1 x_2 + 0.7 x_1 ^2 + x_2 $	$O((x_1 + x_2) \log x_1)$
RedBlackTree	$O((x_1 + x_2) \log x_1)$	$1.5 x_1 + x_1 x_2 + 0.5 x_1 ^2 + x_2 $	$O((x_1 + x_2) \log x_1)$
RedBlackJoin	$O(\log x_1 + \log x_2)$	$ x_1 + x_2 $	$O(\log x_1 + \log x_2)$

Benchmark	Resource Decomposition			
	Bayesian ence	Infer-	Size Bound	Resource Guarded
MergeSort	$d_1 \in O(\log l)$		$ l \leq x $	$2 x d_1$
HeapInsert	$d_1 \in O(\log(h_1 + h_2))$		$ h_1 + h_2 \leq x $	$ x + x d_1$
BalancedBST	$d_1 \in O(\log l)$ $d_2 \in O(\log r)$		$ l \leq x_1 $ $ r \leq x_1 $	$1+0.5d_1 x_1 +0.5 x_1 + x_2 d_2$
RedBlackTree	$d_1 \in O(\log l)$ $d_2 \in O(\log r)$		$ l \leq x_1 $ $ r \leq x_1 $	$ x_1 d_1 + x_1 + x_2 d_2 + x_2 $
RedBlackJoin	$d_i \in O(\log(t_1 + t_2))$		$ t_1 + t_2 \leq x_1 + x_2 $	$d_1 + d_2$

Future Work

Data-driven analysis currently bounds the recursion depth of a function f in terms of the size of its own arguments. To fully automate resource decomposition, we require a static analysis to relate the size of the arguments of f with the size of the global argument x when running $P(x)$.

Extending the program transformation to be able to handle **non-local control flow** will enable cost analysis for programs using exceptions.

References

- [1] Hoffmann, J., Aehlig, K., and Hofmann, M. Multivariate amortized resource analysis. *ACM Trans. Program. Lang. Syst.* 34 (2012),14:1–14:62
- [2] Wang, D., Kahn, D. M., and Hoffmann, J. Resource aware ml, 2020. Available at <https://doi.org/10.1145/3410233>
- [3] Pham, L., Niu, Y., Glover, N., Saad, F., and Hoffmann, J. Integrating resource analyses via resource decomposition. *Proc. ACM Program. Lang.* 9, OOPSLA2 (Oct. 2025)
- [4] Chu, E. The theory and implementation of resource aware ml 2. Master's thesis, Carnegie Mellon University, Pittsburgh, PA, December 2023. Available at <https://www.andrew.cmu.edu/user/ethanchu/raml-thesis.pdf>
- [5] Carpenter, B., Gelman, A., Hoffman, M. D., Lee, D., Goodrich, B., Betancourt, M., Brubaker, M., Guo, J., Li, P., and Riddell, A. Stan: A probabilistic programming language. *Journal of Statistical Software* 76, 1 (2017), 1–32